

การศึกษา ALGORITHM สำหรับการ ทดสอบซอฟต์แวร์ และการออกแบบ PROMPT สำหรับ AI

CP353201 SQA • ปีการศึกษา 1/2569

ณัฐพงศ์ กรรณกิจ 673380038-9

ธนกร ทองศรี 673380040-2

ธนันชัย พันธราช 673380042-8

พีรพัฒน์ ป้องกันยา 673380053-3



GitHub Repository
github.com/Santa1357900/SQA_Project

ที่มาและความสำคัญ

การสร้างกรณีทดสอบด้วยตนเองใช้เวลาและอาจครอบคลุมโปรแกรมได้ไม่ครบถ้วน จึงมีการนำอัลกอริทึมและ AI มาช่วยสร้างกรณีทดสอบอัตโนมัติ อย่างไรก็ตาม ชุดทดสอบที่สร้างขึ้นอาจไม่สามารถใช้งานหรือค้นพบข้อบกพร่องได้จริง โครงการนี้จึงศึกษาการสร้างและเปรียบเทียบกรณีทดสอบจาก DE, ACO และ AI เพื่อประเมินประสิทธิภาพในการตรวจสอบซอฟต์แวร์

วัตถุประสงค์

วัตถุประสงค์ของโครงการ

1. พัฒนา DE และ ACO เพื่อสร้างกรณีทดสอบระดับหน่วยสำหรับโปรแกรม Java ใน Defects4J
2. ประยุกต์ใช้ Claude และ Gemini เพื่อสร้างชุดทดสอบ JUnit
3. เปรียบเทียบประสิทธิภาพทั้ง 4 วิธี พร้อมวิเคราะห์ข้อจำกัดและจัดเก็บหลักฐานการทดลองที่สามารถตรวจสอบและทำซ้ำได้

```
public class UserService {  
    private final UserRepository repository;  
    private final PasswordEncoder encoder;  
  
    public UserService(UserRepository repository,  
        PasswordEncoder encoder) {  
        this.repository = repository;  
        this.encoder = encoder;  
  
        User register(User user) {  
            if (user == null) {  
                throw new IllegalArgumentException("user must not be null");  
            }  
  
            if (repository.findByEmail(user.getEmail()) != null) {  
                throw new IllegalStateException("Email already exists");  
            }  
  
            user.setPassword(encoder.encode(user.getPassword()));  
            user.setCreatedAt(LocalDate.now());  
            repository.save(user);  
        }  
  
        public Optional<User> findById(Long id) {  
            return repository.findById(id);  
        }  
    }  
}
```

ชุดข้อมูลและเกณฑ์ตัดสิน

ใช้ Defects4J ซึ่งมีข้อบกพร่อง 854 ข้อบกพร่องจาก จาก 17 โครงการ Java

เกณฑ์ที่ตรวจพบบักต้อง:

- ผ่านบนโปรแกรมรุ่น Fixed
- ล้มเหลวบนโปรแกรมรุ่น Buggy
- วัด coverage ในคลาสที่ถูกแก้ไข



วิธีสร้างชุดทดสอบ 4 วิธี

- DE: ค้นหาอินพุตด้วย mutation และ crossover
- ACO: สำรวจเมธอดและค่าพารามิเตอร์ผ่านเส้นทางการตัดสินใจ
- Claude: สร้างชุดทดสอบ JUnit ผ่าน KGU IntelSphere
- Gemini: สร้างชุดทดสอบ JUnit ผ่าน KGU IntelSphere

ขั้นตอนการทดลอง

- สร้างชุดทดสอบ
- รันทดสอบกับรุ่น Fixed
- ตัดเทสที่ไม่ผ่าน
- รันเทสที่เหลือกับรุ่น Buggy
- บันทึกผลและวัด coverage

นับว่าตรวจพบบักเมื่อเทสผ่านบน Fixed และล้มเหลวบน Buggy

```
public class UserService {  
    private final UserRepository repository;  
    private final PasswordEncoder encoder;  
  
    public UserService(UserRepository repository,  
        PasswordEncoder encoder) {  
        this.repository = repository;  
        this.encoder = encoder;  
    }  
  
    public User register(User user) {  
        if (user == null) {  
            throw new IllegalArgumentException("user must not be null");  
        }  
        if (repository.findById(user.getId()) != null) {  
            throw new IllegalStateException("Email already exists");  
        }  
        user.setPassword(encoder.encode(user.getPassword()));  
        user.setCreatedAt(LocalDate.now());  
        repository.save(user);  
        return user;  
    }  
  
    public Optional<User> findById(Long id) {  
        return repository.findById(id);  
    }  
}
```

1. สุ่มประชากร

สุ่มประชากรเริ่มต้นแต่ละตัวคือชุด input ของเมรอต

2. Mutation

สร้าง Mutation จากสูตร

$$v = x1 + F \times (x2 - x3)$$

3. Crossover

ผสมค่าใหม่กับอินพุตเดิมตามความน่าจะเป็น crossover

4. Selection

เก็บตัวที่ fitness ดีกว่าไว้ในรุ่นถัดไป

Differential Evolution (DE)

เป็นอัลกอริทึมเชิงวิวัฒนาการที่มีจุดเด่นในด้านการกลายพันธุ์ที่ใช้ผลต่างของเวกเตอร์มาปรับอินพุต ใช้รูปแบบ DE/rand/1/bin ค้นหา input ของเมรอต มี default config ดังนี้

- population 16, budget 512
- mutation $F = 0.7$, crossover $CR = 0.9$
- input range 1000, seed 2026

1. เลือกเมธอด

มดเลือกเมธอดหรือ constructor ของคลาสเป้าหมาย

2. เลือกค่า input

เลือกวิธีสร้าง object และค่าพารามิเตอร์ เน้นค่าชอบ

3. วัด coverage

รันบน buggy แล้วนับ probe ใหม่ของ JdCoCo

4. ปรับฟีโรโมน

เมื่อไม่สามารถหาเส้นทางต่อได้ ค่าฟีโรโมนจะค่อยๆลดลง และจะทำให้เกิดเส้นทางใหม่

Ant Colony Optimization (ACO)

เหตุผลที่เลือกอัลกอริทึมนี้ เพราะ การลองเมธอดและค่าพารามิเตอร์มีทางเลือกเยอะกว่า และ ACO จะช่วยจัดลำดับว่าควรเลือกทางไหนก่อน จากการเลียนแบบมดโดยใช้ ฟีโรโมน และ heuristic ในการตัดสินใจเลือกผลลัพธ์ที่ดี โดย มดแต่ละตัวจะสร้างเส้นทางหนึ่งตัวโดยเดินบนกราฟการตัดสินใจ

โครงงานนี้ ใช้ Config ตามนี้

- ใช้มด 20 ตัวต่อรอบ 150 รอบ
- $\alpha = 1.0$, $\beta = 2.0$ ใช้ปรับน้ำหนักระหว่างเส้นทางที่เคยให้ผลดี กับ ข้อมูลชี้นำในการเลือกทาง
- อัตราระเหย = 0.25 ฟีโรโมนเก่าลดลงในแต่ละรอบ เพื่อให้การค้นหาไม่ยึดติดกับทางเลือกเดิมมากเกินไป

★ Gemini และ Claude ✨

1. ใช้ gemini-3.5-flash-lite 2. Claude ใช้ Sonnet 5
ผ่าน KKU IntelSphere สร้าง JUnit 4 จาก prompt

✓ ดึงซอร์สคลาสเป้าหมายจาก Defects4J

✓ ประกอบ prompt จาก template

✓ เรียก API ให้สร้างไฟล์เทส

✓ รับเทสแบบ buggy และ fixed

✓ วัด coverage และสรุปผล



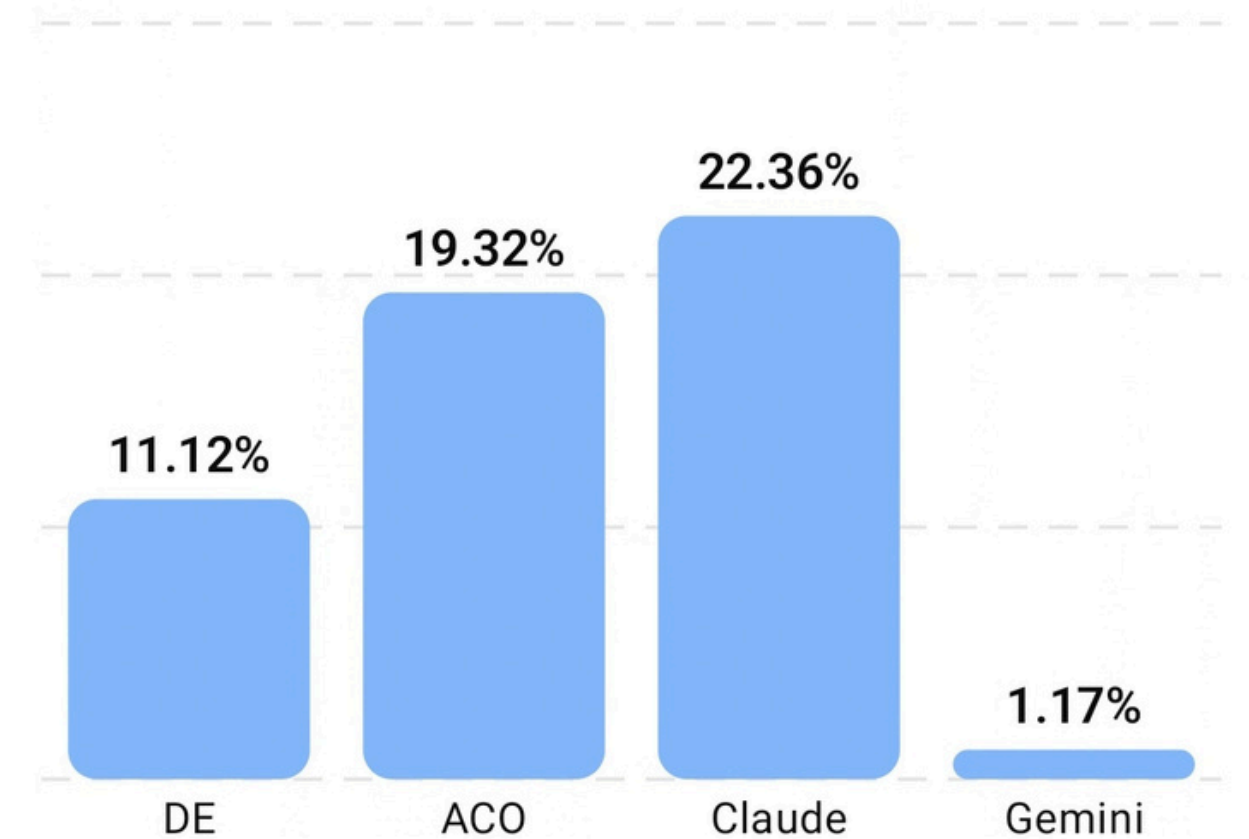
ผลรวมจาก Defects4J

จำนวนรายการที่ประเมินผลได้ของแต่ละวิธีไม่เท่ากัน

วิธี	ประเมินได้ / 854	ตรวจพบบัก	Fault Detection Rate ที่ประเมินได้	Fault Detection Rate ทั้งหมด
DE	701	95	13.55%	11.12%
ACO	844	165	19.55%	19.32%
Claude	697	191	27.40%	22.36%
Gemini	165	10	6.06%	1.17%

Fault Detection Rate ทั้งหมด

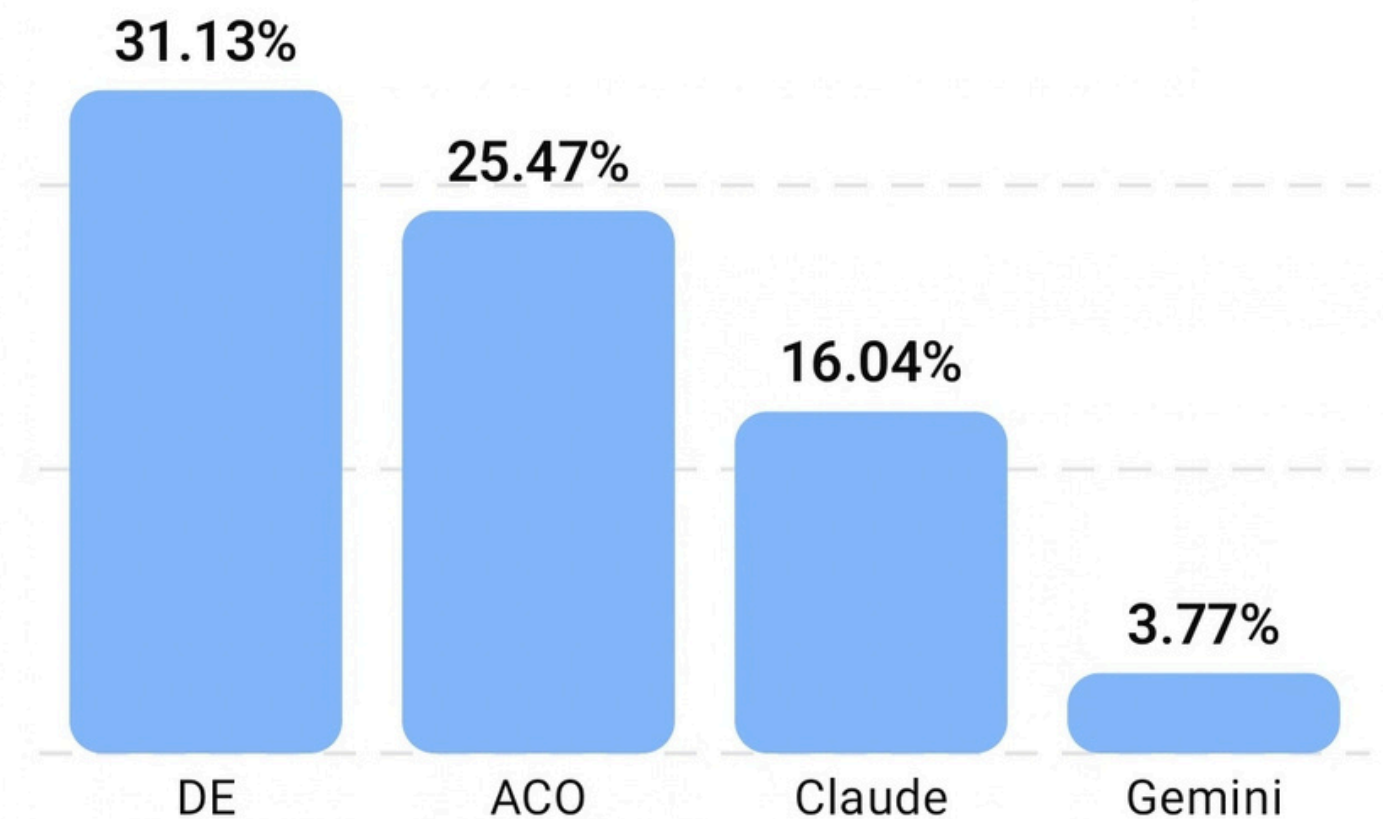
อัตราการตรวจพบบักเมื่อเทียบกับ Defects4J ทั้งหมด 854 รายการ



เปรียบเทียบเป้าหมายเดียวกัน จาก Math 106 บั๊ก

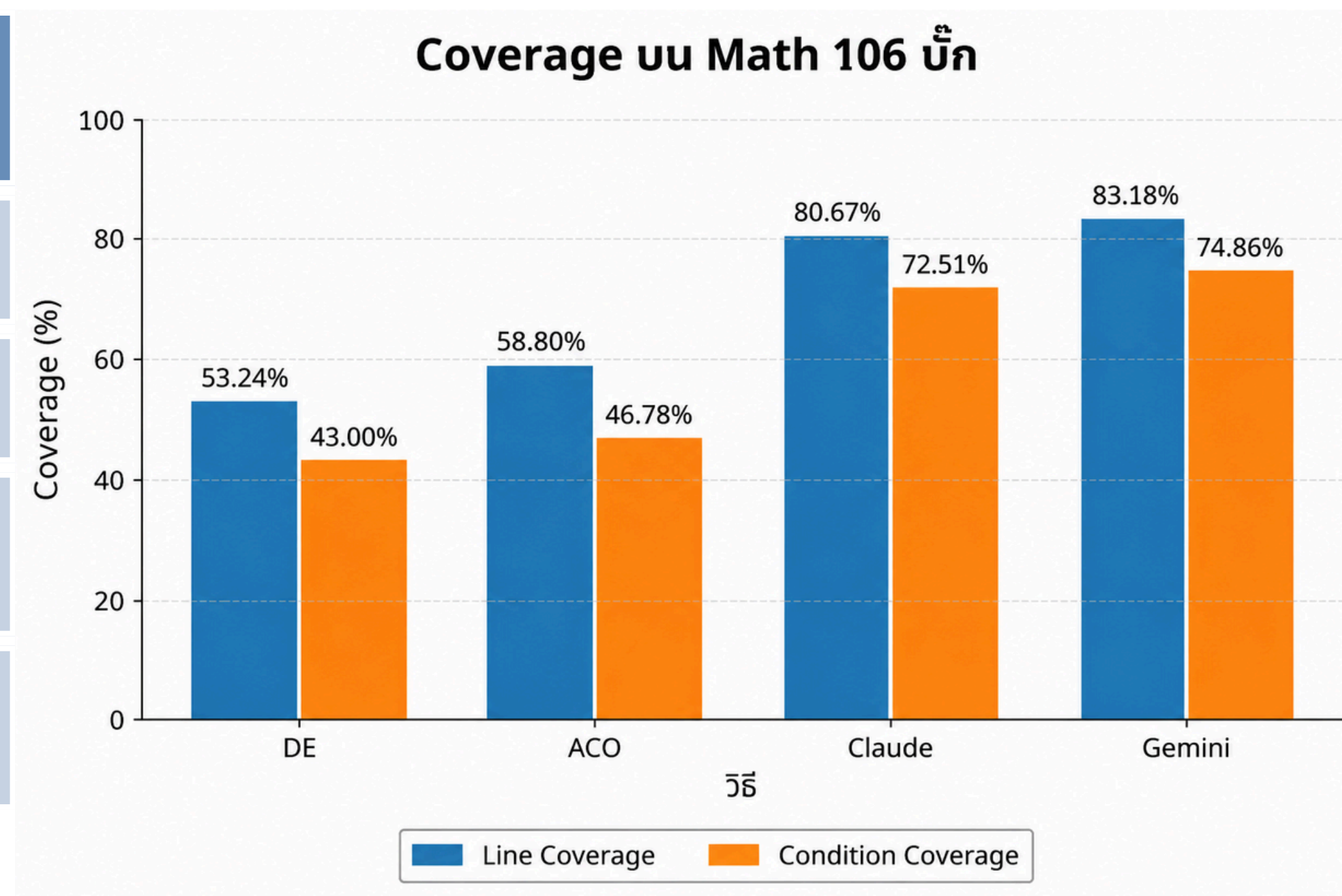
วิธี	ตรวจพบบั๊ก	ตัดสินใจได้	ตรวจพบจากเป้าหมาย 106
DE	33	103	31.13%
ACO	27	103	25.47%
Claude	17	96	16.04%
Gemini	4	34	3.77%

Fault Detection Rate จากเป้าหมาย 106 บั๊ก
 เปรียบเทียบอัตราการตรวจพบบั๊กของ DE, ACO, Claude และ Gemini จากเป้าหมายเดียวกัน 106 บั๊ก



Coverage uu Math 106 ปีก

วิธี	Line coverage	Condition coverage	จำนวน
DE	53.24%	43.00%	96
ACO	58.80%	46.78%	103
Claude	80.67%	72.51%	103
Gemini	83.18%	74.86%	76



Coverage ไม่เท่ากับการตรวจพบบั๊ก

Coverage จะแสดงว่าทดสอบเข้าถึงโค้ดส่วนไหน
แต่ การตรวจพบบั๊กแสดงว่าทดสอบแยกพฤติกรรมของรุ่น Buggy กับ
Fixed ได้หรือไม่

แม้ Gemini มี Line coverage เฉลี่ย 83.18% ใน 76 รายการ
แต่ตรวจพบบั๊กเพียง 4 รายการจากเป้าหมาย Math 106 รายการ
และมีผลตัดสิน fault เพียง 34 รายการ

**จึงควรพิจารณา coverage ควบคู่กับคุณภาพของ assertion
และจำนวนผลที่ประเมินได้**

```
public class UserService {  
    private final UserRepository repository;  
    private final PasswordEncoder encoder;  
  
    public UserService(UserRepository repository,  
        PasswordEncoder encoder) {  
        this.repository = repository;  
        this.encoder = encoder;  
    }  
  
    public User register(User user) {  
        if (user == null) {  
            throw new IllegalArgumentException("user must not be null");  
        }  
        repository.findByEmail(user.getEmail()) != null) {  
            throw new IllegalStateException("Email already exists");  
        }  
        user.setPassword(encoder.encode(user.getPassword()));  
        user.setCreatedAt(LocalDate.now());  
        repository.save(user);  
        return user;  
    }  
  
    public Optional<User> findById(Long id) {  
        return repository.findById(id);  
    }  
}
```

ข้อจำกัดและปัญหาที่พบ



การทดลองเพียง 1 รอบ จึง
ยังไม่เห็นความแปรปรวนจาก
การสุ่มหรือการเรียกโมเดล



โควตา KKU token
มี จำกัดต่อวัน



การใช้พรอมพ์ในการสร้าง
Test ของ Gemini กับ
Claude ไม่เหมือนกัน



Compile error ของ
Claude คอมไพล์ไม่ผ่าน
144 บั๊ก โดย 80 บั๊กอยู่ใน
Closure เพราะโมเดลเดา
API ผิด



วิเคราะห์และสรุป



เมื่อพิจารณาร่วมกับวิธีทำงาน DE และ ACO มีค่ากำหนดการค้นหาที่ตรวจสอบได้ โดย ACO ให้ผล
ประเมินครบกว่าใน Math ส่วน Claude ตรวจพบข้อบกพร่องมากที่สุดแต่ต้องคัดกรองชุดทดสอบที่สร้างจากโมเดล
Gemini ให้ค่า coverage สูงในรายการที่วัดได้แต่ยังมีผลตัดสินการตรวจพบไม่ครบจำนวนมาก ข้อจำกัดของแต่ละ
วิธีจึงสัมพันธ์กับทั้งวิธีสร้างข้อมูลนำเข้า คุณภาพของ assertion และความพร้อมของกระบวนการตรวจสอบ

Demo

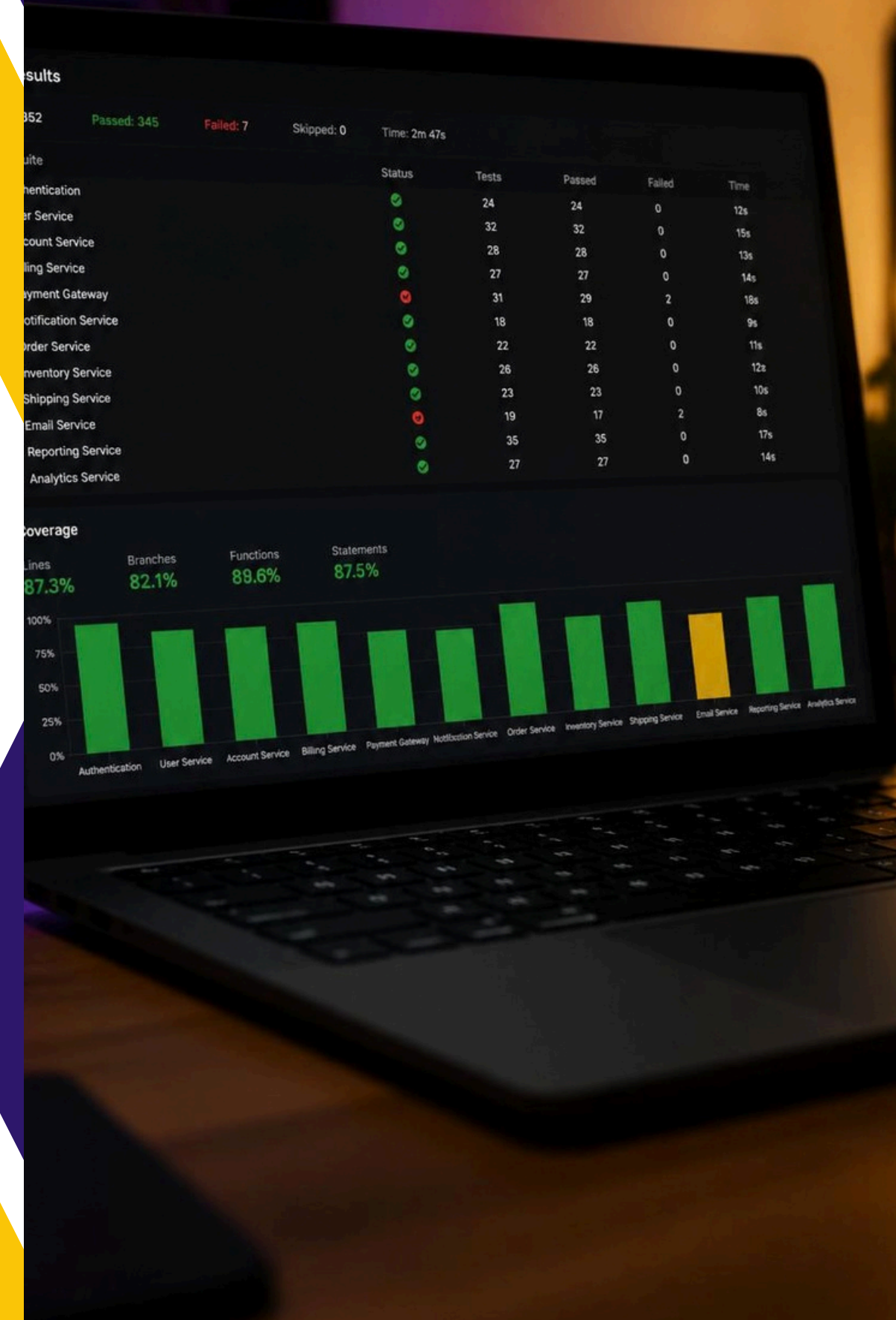
สาธิตการสร้างและวัดผล unit test ของบ๊ีก Lang-1 ด้วย Claude ตั้งแต่
สร้างเทสจนได้ผลลัพธ์

1. สร้างเทส

2. รันแบบ fixed

3. รันแบบ buggy

4. ดู result.json



THANK YOU

FOR YOUR ATTENTION



GitHub Repository
github.com/Santa1357900/SQA_Project